NTU-114-1-ADL-HW3

Chia-Chun Liang

Data Science Degree Program

National Taiwan University

Student ID: d14948004

Email: d14948004@ntu.edu.tw

Oct 2025

Task Description and Overview

The objective of this assignment is shown in Figure 1.

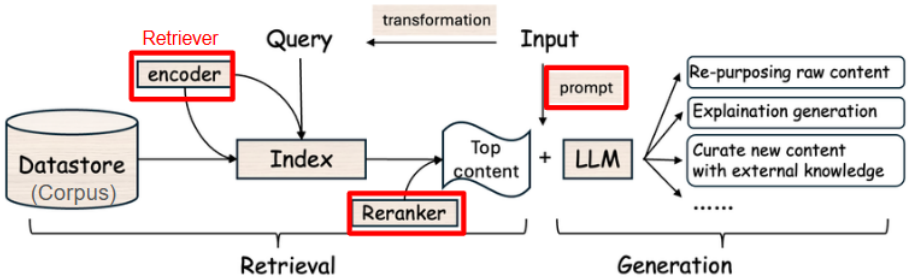

Figure 1: An overview of the whole task process.

Report - Q1: Retriever & Reranker Tuning (5%)

Retriever Training Process (2.5%) and Reranker Training Process (2.5%).

Both should include, but are not limited to, the following:

- Clearly describe how the training data is constructed (e.g., anchor, positive sampling, and negative sampling strategies).
- Specify and explain the loss function used for training.
- List the hyperparameters adopted in your experiments.

- Provide a training loss curve with at least 5 data points to illustrate the training process. (You can use any package to plot)
  - To receive full credits, you should still describe your experiments for both the retriever and reranker training processes, even if you ultimately decide to use the base model as your submission.

Train/Validation data construction: Selection Initially, I used the entire `train.txt` file for model learning, while `test_open.txt` was used for evaluation. However, the evaluation results were unsatisfactory, with Recall@10 remaining under 0.69, as shown in Figure 2, and the reranker performing similarly, with its MRR@10 also dropping to about 0.20.

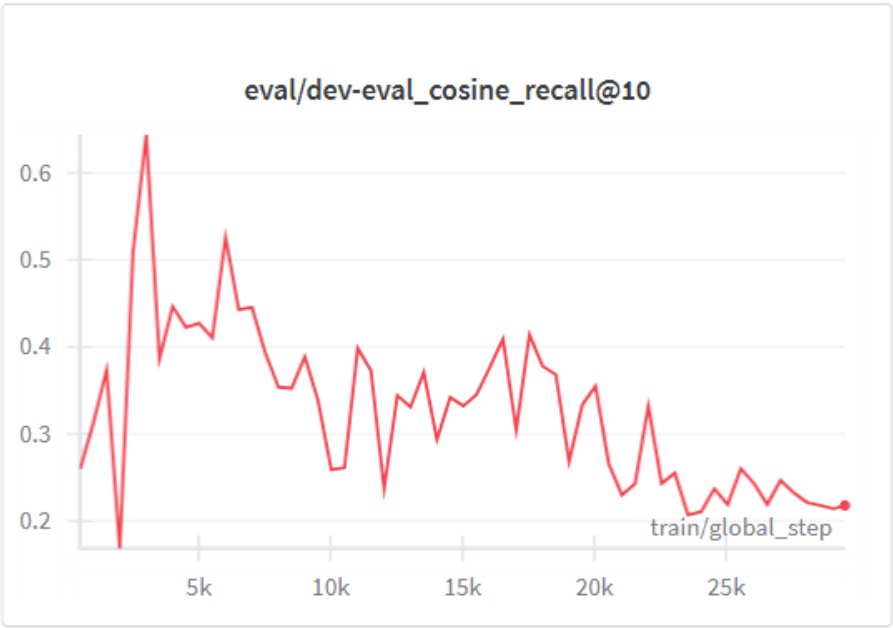

Figure 2: Initial evaluation result showing low Recall@10 (≤ 0.69) when using the imbalanced `test_open.txt` as the validation set.

To understand this behavior, I conducted a detailed inspection of the dataset distribution and found significant imbalance and sparsity issues in the evaluation data.

Specifically, as shown in Table 1, although about one quarter of the queries in `test_open.txt` (24.86%) contain at least one positive passage, the overall positive-to-negative ratio remains extremely low at only 4.97% ($\frac{831}{16,715}$). This imbalance means that most passages in the evaluation set are irrelevant, making it difficult for the model to achieve stable recall and ranking performance.

In contrast, the training data `train.txt` is much more balanced, with each one of the 31,526 queries containing at least one positive example and an overall positive ratio of 20.0%.

Table 1: Comparison of data distribution between `test_open.txt` and `train.txt`.

Statistic	**test_open.txt**	**train.txt**
Total queries	3,343	31,526
Queries with positive samples	831 (24.86%)	31,526 (100.00%)
Total positives	831	31,526
Total negatives	15,884	126,104
Positive ratio	4.97%	20.00%

Given the strong data imbalance in `test_open.txt`, I constructed a more representative validation set from the training data itself. The original training set contained 31,526 valid queries, each with at least one positive passage. Some queries were labeled as "CANNOTANSWER," indicating that no positive passage is available.

To ensure consistent label distributions in both training and validation sets, I split the queries in 8-to-2-ratio. As for those "CANNOTANSWER" ones, I chose them out and also split and merge to the 8-to-2-ratio train-val dataset. The resulting subsets are:

- Training set: 25,221 queries
- Validation set: 6,305 queries

This splitting procedure was implemented via a custom script (`split_train_val.py`) and produced the following final files:

- `data/train_split.txt`
- `data/train_val.txt`

By re-balancing the validation data in this manner, the model was able to learn from a more stable and representative data distribution, leading to more consistent training dynamics. Consequently, the retriever's Recall@10 and the reranker's MRR@10 exhibited more steady improvements during subsequent training.

I also found a related discussion in the Hugging Face blog post [Kue24], which supports the discussion above. The article points when using *MultipleNegativesRankingLoss* (corresponding to my retriever-training stage), many supposedly "negative" samples may actually be relevant (i.e., false negatives) due to incomplete labeling. Such imbalance can mislead the model and degrade retrieval performance. This aligns with my observation that the original `test_open.txt` had very few positive samples, leading to unstable evaluation results.

Different training strategy from Retriever To Reranker. Starting from the Retriever training strategy, following the guideline from the Hugging Face tutorial on training Sentence Transformers [Esp22], there are four common input data formats for sentence-transformer training. Among them, the second, third, and fourth formats are suitable for retrieval tasks. I adopt the **second format**, where each training example consists of a single query–positive pair. This choice aligns with the goal of maximizing recall in the retriever stage of the RAG pipeline.

Here is how it goes: firstly the corpus (`data/corpus.txt`) and qrels (`data/qrels.txt`) are loaded to construct training and validation examples. The corpus is stored as a dictionary mapping document IDs to passage texts, each prefixed with "`passage:`", as the following:

$$\texttt{corpus}[doc_id] = \text{"passage: <text>"}.$$

The qrels are loaded as a dictionary mapping each query ID (qid) to a single relevant document ID (pos_id):

$$\texttt{qrels}[qid] = pos_id.$$

This ensures that each query can be paired with at least one positive passage for training.

For the **retriever**, each training dataset consists of a query–positive pair. The query text is prefixed with "`query:`" and paired with a relevant passage from the corpus:

```
InputExample(texts=[query, pos]).
```

Negative examples are not explicitly sampled; instead, all other passages within the same mini-batch are treated as negatives under the *MultipleNegativesRankingLoss*. This in-batch negative sampling strategy ensures efficient contrastive learning without requiring explicit negative mining.

Specifically, this approach leverages the *in-batch negative sampling* mechanism: for each query, all other positive passages within the same mini-batch (set to 128 in this work) are treated as negatives. Such contrastive learning design encourages the model to bring semantically relevant passages closer while pushing irrelevant ones apart, thereby improving retrieval robustness and recall.

As for the **reranker**, the training and validation data were initially constructed from the original `train.txt`. I later realized that this setup could not ensure the fine-tuned retriever would be able to distinguish the correct passage from other, more similar negative samples. While the model performed reasonably well when evaluated on `test_open.txt`, its performance dropped when tested using `inference_batch.py`. This discrepancy arises because, in the actual RAG inference scenario, the reranker must select the gold passage from a set of highly similar candidate passages rather than from clearly irrelevant ones. Therefore, the reranker's training data should be regenerated using the fine-tuned retriever, which better reflects the real inference conditions (see *fine-tune-v-1*, Table 10).

After fine-tuning the retriever, I reconstructed the reranker's training dataset to better reflect real inference conditions. Specifically, for each query, the gold passage returned by the retriever is treated as the positive sample, and an additional 9 passages are sampled from the top-retrieved candidates as hard negatives. This setup ensures that the reranker learns to distinguish the correct passage from other highly similar passages, which mirrors the actual RAG inference scenario where the retriever provides a top-10 candidate set and the reranker must select the gold passage among them.

Hyperparameters adopted for training For the **retriever**, the assigned `intfloat/multilingual-e5-small` is trained with the following hyperparameters:

- training batch size = 128
- learning rate = 1×10^{-5}
- training epochs = 5
- use_fp16 = True
- max_seq_length=512
- eval_steps = 40

The validation is helped by an `InformationRetrievalEvaluator` over the validation set, measuring Recall@1, Recall@5, Recall@10, and MRR@10. The best model was selected based on the highest Recall@10.

For the **reranker**, (`cross-encoder/ms-marco-MiniLM-L-12-v2`)is trained with the following hyperparameters:

- training batch size = 16
- learning rate = 2×10^{-5}
- warmup_steps = 50
- training epochs = 3
- use_fp16 = True
- max_seq_length=512
- eval_steps = 40

Loss function The **retriever** model employs the `MultipleNegativesRankingLoss`, which encourages a high cosine similarity between the query and its corresponding positive passage while penalizing similarity to other (implicitly negative) passages in the same batch. Formally, for a given query q and positive passage p^+, the loss is defined as:

$$\mathcal{L} = -\log \frac{\exp(\text{cos-sim}(q, p^+)/\tau)}{\sum_{i=1}^{B} \exp(\text{cos-sim}(q, p_i)/\tau)},$$

where cos-sim$(\cdot)$ denotes cosine similarity, following the implementation in the Sentence-Transformers library. τ is a temperature parameter, and B is the batch size.

In contrast, the **reranker** model is trained using a regression objective, where the predicted relevance score $\hat{y}$ is optimized against the ground truth label y using Mean Squared Error (MSE). I implement the loss using PyTorch's `torch.nn.functional.mse_loss`. Formally, for a batch of N query-passage pairs, the loss is computed as:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2,$$

where $y_i \in \{0, 1\}$ indicates whether the i-th document is relevant (1) or not relevant (0). This formulation allows the model to output a continuous relevance score, which can then be used for ranking during inference.

Training Loss Curves Figure 3 and Figure 4 show the training loss curves for the retriever and reranker, respectively. The retriever achieves a final training loss of 0.0046, indicating stable convergence, while the reranker reaches 0.0534, consistent with the more complex cross-encoder architecture and ranking task.

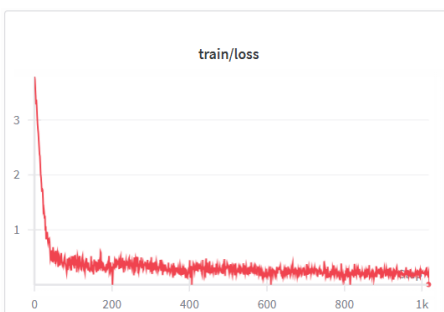

Figure 3: Training loss curve of the retriever model. Final loss: 0.0046.

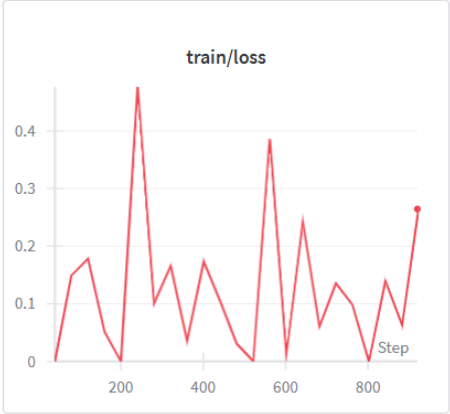

Figure 4: Training loss curve of the reranker model. Final loss: 0.2645.

Evaluation Curves The evaluation curves of the retriever and reranker demonstrate their performance over training steps. For the retriever, Figures 5–7 plot Recall@1, Recall@5, and Recall@10 on the validation set, showing gradual improvement and stabilization. The retriever achieves the following metrics on the validation set (`train_val.txt`): Recall@1 = 0.6185, Recall@3 = 0.9242, Recall@5 = 1.0000, Recall@10 = 1.0000, MRR = 0.7741, MAP = 0.7741.

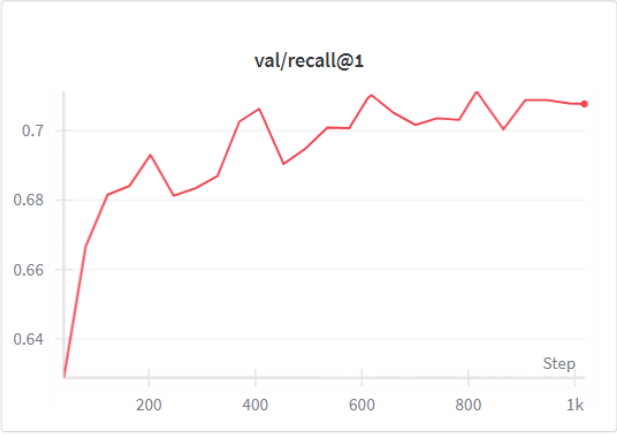

Figure 5: Retriever evaluation curve for Recall@1 over training steps.

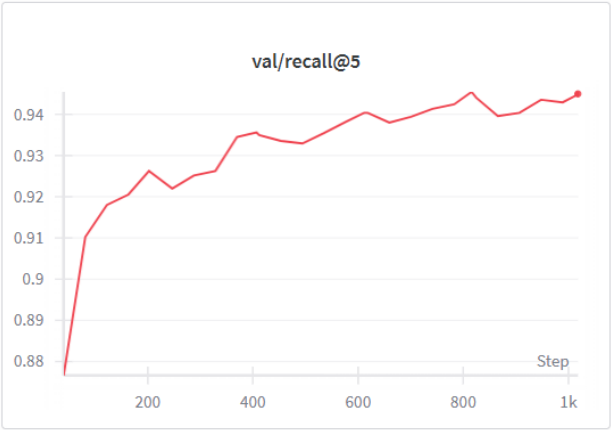

Figure 6: Retriever evaluation curve for Recall@5 over training steps.

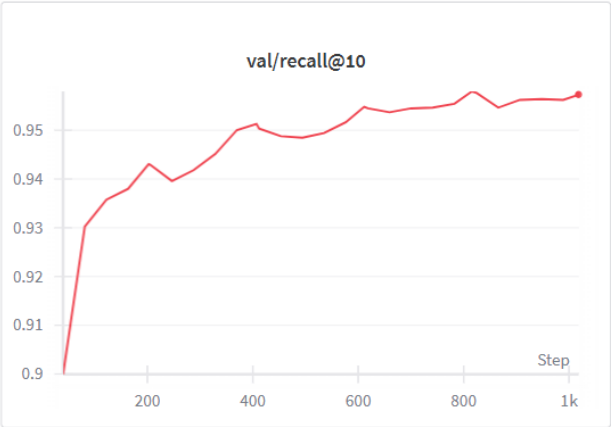

Figure 7: Retriever evaluation curve for Recall@10 over training steps.

For the reranker, Figure 8 shows the MRR@10 curve over training, reaching a best value of 8767. These curves confirm that the reranker effectively learns to promote relevant documents to the top ranks, complementing the retriever's coarse ranking.

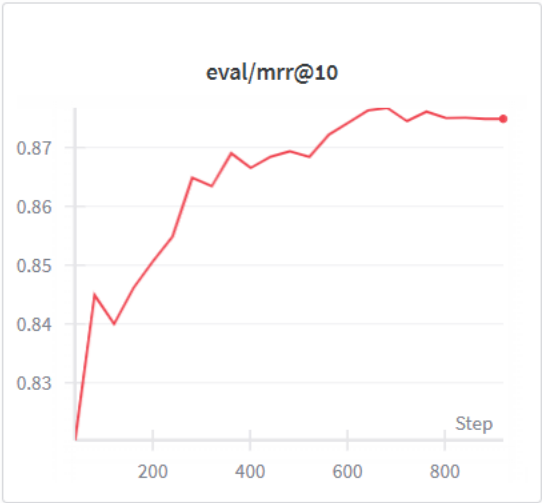

Figure 8: Reranker evaluation curve (MRR@10) over training steps. Best score on *train_val.txt* achieved MRR@10: 0.9828.

Report - Q2: Prompt Optimization (3%)

Description

- Provide a detailed explanation of how you designed your prompt. (1.5%)
- Present at least three different prompts you experimented with, along with their inference performance, to demonstrate the effectiveness of your optimization approach. (1.5%)

First Try I started with a straightforward prompt as the initial baseline. The system and user prompts were defined as shown in Table 2.

System Prompt	`You are a helpful AI assistant. Your task is to answer questions based on the provided context passages.`
User Prompt	`Context Information:\n{context_text}\nQuestion: {query}\n\nBased on the context passages above, please provide a clear and accurate answer to the question. If the answer cannot be found in the provided context, please state that explicitly.`

Table 2: The design of the initial prompt (`prompt 1`) used in the first experiment, including both the system and user components.

Recall that the inference pipeline implemented in `inference_batch.py` follows the sequence *Load data* → *Retrieve* → *Rerank* → *Generate* → *Evaluate*. Upon closer inspection, I found that most improvements could be made during the *generation* stage. The simple prompt design caused the LLM (Qwen/Qwen3-1.7B) to produce overly verbose and explanatory responses instead of concise, span-based answers. As a result, the generated outputs often deviated from the expected format, as illustrated in Table 3.

Query	`Aside from incorporating, being president of the MLB association, and his work with free agency, what else did Bob Feller's do in his later life?`
Gold Answer	`Feller and his wife, Virginia Winther, had three sons, Steve, Martin, and Bruce.`
Generated Answer	`<think>\n\n</think>\n\nBased on the provided context passages, there is no information about Bob Feller (note: the name in the question is Bob Feller,ƀut the context refers to Sid Fellerände Jack Feller) doing anything else aside from incorporating, being president of the MLB association, and his work with free agency. Therefore, the answer is:\n\nThe context does not provide information about Bob Feller's later life beyond the mentioned activities.`

Table 3: An example output generated with the fine-tuned retriever and pre-trained reranker.

Upon revisiting the task description, I noticed the explicit instruction:

> "The answer can be an exact span of the positive passage or `CANNOTANSWER` in both train and test data."

This clarified that the verbose, explanatory outputs violated the task constraints. Consequently, I redesigned the prompt to explicitly enforce either span extraction or the `CANNOTANSWER` format.

In the post-processing step (`parse_generated_answer`), in addition to removing Qwen's BOS/EOS tokens (`<|im_start|>`, `<|im_end|>`), I also stripped out the `<think>\n\n</think>` tags to clean the final outputs.

Second prompts To address the verbosity and inconsistency observed in the first setup, the second configuration (`prompt 2`) explicitly constrained the model's behavior to *span extraction*. In this design, the system prompt redefined the model as **an extraction model** rather than *a general question-answering assistant*. The user prompt introduced step-by-step instructions to enforce a strict-copying-strategy from the provided passages. And I added the rule following from the hint noted from the assignment: that if the answer was not

explicitly present, the model must output the literal string CANNOTANSWER. The system and user prompts used in this version are summarized in Table 4.

In the second configuration, system and user prompts were defined as shown in Table 4.

System Prompt	You are an extraction model.\n Your only job is to copy the exact answer text from the provided passages. \nDo not explain, summarize, rephrase, guess, or include external knowledge.\n If no answer is clearly supported by the context, output exactly: `CANNOTANSWER`.\n
User Prompt	Context Information:\n{context_text}\nQuestion: {query} Instructions: 1. Copy the exact answer phrase directly from the passages above. 2. Do NOT paraphrase, expand, or add explanations. 3. If the answer is not in the context, output exactly: "CANNOTANSWER". 4. Your answer must be one continuous text span or CANNOTANSWER. Answer:

Table 4: The design of the second prompt (prompt 2), including both the system and user components.

These modifications progressively guided the model toward producing concise outputs or directly outputting CANNOTANSWER. Empirically, this led to a marked reduction in verbose responses and hallucinated reasoning, resulting in shorter, span-based predictions that aligned more closely with the gold annotation format. However, it also slightly increased the proportion of CANNOTANSWER outputs, indicating a more conservative extraction behavior. Specifically, while only **690** instances in the gold labels were CANNOTANSWER, the model produced **2,123** such outputs (in the scenario of pretrained reranker), suggesting an overcautious extraction behavior.

The Following Prompts To further improve the model's ability to perform accurate *extraction*, we introduced demonstration-based prompts in Prompt 3 and Prompt 4. Specifically, a demonstration generated by ChatGPT was added based on a training example, *"Was the album successful?"*, taken from train_val.txt. In Prompt 3, the demonstration was inserted into the system prompt, while in Prompt 4, it was included in the user prompt. However, both approaches unexpectedly led to degraded performance. These results suggest that the demonstrations may have introduced unintended biases, thereby distracting the model from factual extraction rather than improving it.

Furthermore, after consulting with AI, an additional *key principle* was proposed to prevent the model from overusing the response "CANNOTANSWER." Although it initially appeared to make sense, it instead resulted in worse performance. Another strategy introduced a form of *hierarchical confidence assess-*

ment, aiming to make the model evaluate its certainty step by step; however, this approach performed even worse in practice.

As shown in Table 5, the proportion of generated "CANNOTANSWER" responses increases substantially from `Prompt 2` to `Prompt 6`, corresponding to a consistent decline in Bi-Encoder cosine similarity. These observations indicate that emphasizing uncertainty control or multi-stage reasoning can make the model overly conservative in its judgment, reducing its willingness to extract answers and consequently degrading overall retrieval quality.

Table 5: Statistics of `CANNOTANSWER` generation across different prompt configurations (using the **vanilla reranker**). All results are computed over a total of 3342 test cases. `CANNOTANSWER (Gen)` denotes the number of times the model generated `CANNOTANSWER`, while `CANNOTANSWER (Gold)` indicates the number of ground-truth unanswerable samples. The last two columns report the percentage of cases where the model produced an extractive answer and the average Bi-Encoder cosine similarity.

Prompt	`CANNOTANSWER (Gen)`	`CANNOTANSWER (Gold)`	Model extractable (%)	Bi-Encoder CosSim
Prompt 1	0	690	100.00	0.3661
Prompt 2	2123	690	36.48	0.3478
Prompt 3	2196	690	34.29	0.3270
Prompt 4	2286	690	31.60	0.3297
Prompt 5	2645	690	20.86	0.3178
Prompt 6	3113	690	6.85	0.2699

The configurations discussed above were defined as shown in Table 6 to Table 9.

System Prompt	`You are an extraction model.\n Your only job is to copy the exact answer text from the provided passages. \nDo not explain, summarize, rephrase, guess, or include external knowledge.\n If no answer is clearly supported by the context, output exactly: `CANNOTANSWER`.\n\n Example:\n Passage: "The album gained critical acclaim and achieved gold status." \n Query: "Was the album successful?" \n CORRECT: "The album gained critical acclaim and achieved gold status." \n WRONG: "Yes, it was a massive critical and commercial success." \n WRONG: "The album, released in 1988, was critically acclaimed."`
User Prompt	`Context Information:\n{context_text}\nQuestion: {query} \n\nInstructions: 1. Copy the exact answer phrase directly from the passages above. 2. Do NOT paraphrase, expand, or add explanations. 3. If the answer is not in the context, output exactly: "CANNOTANSWER". 4. Your answer must be one continuous text span or CANNOTANSWER. Answer:`

Table 6: The design of the third prompt (`Prompt 3`), where a demonstration example is inserted into the *system prompt*.

System Prompt	`You are an extraction model.\n Your only job is to copy the exact answer text from the provided passages. \nDo not explain, summarize, rephrase, guess, or include external knowledge.\n If no answer is clearly supported by the context, output exactly: `CANNOTANSWER`.\n`
User Prompt	`Context Information:\n{context_text}\nQuestion: {query} \n\nInstructions: 1. Copy the exact answer phrase directly from the passages above. 2. Do NOT paraphrase, expand, or add explanations. 3. If the answer is not in the context, output exactly: "CANNOTANSWER". 4. Your answer must be one continuous text span or CANNOTANSWER. \n\nExample:\n Passage: "The album gained critical acclaim and achieved gold status."\n Query: "Was the album successful?" \n CORRECT: "The album gained critical acclaim and achieved gold status." \nWRONG: "Yes, it was a massive critical and commercial success." \n WRONG: "The album, released in 1988, was critically acclaimed."\n\nAnswer:`

Table 7: The design of the fourth prompt (`Prompt 4`), where a demonstration example is inserted into the **user prompt**.

System Prompt	`You are an extraction model.\n Your only job is to copy the exact answer text from the provided passages. \nDo not explain, summarize, rephrase, guess, or include external knowledge.\n If no answer is clearly supported by the context, output exactly: CANNOTANSWER.\n\n Key principle: Prioritize extracting actual text over refusing to answer.`
User Prompt	`Context Information:\n{context_text}\nQuestion: {query} \n\nInstructions: 1. Copy the exact answer phrase directly from the passages above. 2. Do NOT paraphrase, expand, or add explanations. 3. If multiple phrases answer the question, choose the most direct one. 4. Only output \"CANNOTANSWER\" if no relevant information exists. 5. Your answer must be one continuous text span or CANNOTANSWER. \n\nAnswer:`

Table 8: The design of the fifth prompt (`Prompt 5`), incorporating the **key principle** to reduce unnecessary "CANNOTANSWER" outputs.

System Prompt	`You are an extraction model.\n Your only job is to copy the exact answer text from the provided passages. \n Do not explain, summarize, rephrase, guess, or include external knowledge.\n Assess confidence: HIGH (clear answer), MEDIUM(partial match), LOW (uncertain). \nOutput CANNOTANSWER only for LOW confidence cases.\n\n\n Confidence rules:\n- HIGH: Answer explicitly stated, no ambiguity. \n- MEDIUM: Answer implied or requires minor inference. \n- LOW: Answer vague or contradictory → output CANNOTANSWER`
User Prompt	`Context Information:\n{context_text}\nQuestion: {query} \n\nInstructions: 1. Copy the exact answer phrase directly from the passages above. 2. Do NOT paraphrase, expand, or add explanations. 3. Assess your confidence:\n - HIGH confidence: Extract and output the phrase\n - MEDIUM confidence: Extract but note it's partial\n - LOW confidence: Output CANNOTANSWER 4. If the answer is not in the context, output exactly: \"CANNOTANSWER\".`

Table 9: The design of the sixth prompt (`Prompt 6`), which applies a **hierarchical confidence assessment** to encourage step-wise extraction.

As shown in Table 10, we compare the RAG reranker performance across three configurations: (1) the original **vanilla** reranker (`cross-encoder/ms-marco-MiniLM-L-12-v2`), (2) the reranker fine-tuned on the same train/validation split as the retriever (**Fine-tune-v1**), and (3) the reranker retrained with a reallocated dataset (**Fine-tune-v2**). Each configuration is evaluated under six different prompt variants (Prompt 1–6), while keeping the retriever fixed (Recall@10 = 0.8223).

The results reveal some interesting observations as the following: First, when the reranker is underperforming (as in the vanilla setup), no prompt design can *fully* recover the retrieval quality. In addition, the overall cosine similarity consistently declines from Prompt 1 to Prompt 6. Second, even with a stronger reranker (Fine-tune-v2), poor prompt phrasing still leads to notable degradation, confirming that prompt quality remains a limiting factor. Finally, across all reranker settings, the performance trend from Prompt 1 to Prompt 6 remains consistent, suggesting that prompt robustness is largely independent of reranker training quality but follows a stable hierarchy of effectiveness.

Reflection on Prompt1 to Prompt6. We first compared these 6 prompting configurations. Note that these configurations range from open-ended to extraction-oriented limitation. Among them, `Prompt1` adopted a general *helpful assistant* instruction, allowing semantic reasoning and flexible synthesis across

Table 10: RAG reranker results across configurations and prompts (Recall@10 = 0.8223). Public baseline values are underlined, and the highest CosSim score is highlighted in **bold**. Each configuration includes six prompt variants. Metrics include Recall@10, MRR@10 after reranking, and Bi-Encoder CosSim.

Config.	Prompt	Recall@10	MRR@10	CosSim
Vanilla	1	0.8223	0.7030	0.3661
	2	0.8223	0.7030	0.3478
	3	0.8223	0.7030	0.3270
	4	0.8223	0.7030	0.3297
	5	0.8223	0.7030	0.3178
	6	0.8223	0.7030	0.2699
w/ Fine-tune-v1 Reranker	1	0.8223	0.5438	0.3614
	2	0.8223	0.5438	0.3407
	3	0.8223	0.5438	0.3189
	4	0.8223	0.5438	0.3261
	5	0.8223	0.5438	0.3052
	6	0.8223	0.5438	0.2695
w/ Fine-tune-v2 Reranker	1	0.8223	0.7088	**0.3681**
	2	0.8223	0.7088	0.3513
	3	0.8223	0.7088	0.3279
	4	0.8223	0.7088	0.3307
	5	0.8223	0.7088	0.3169
	6	0.8223	0.7088	0.2703

multiple passages. Though this may lead to hallucination, paraphrase or extra information, it turned out that the performance evaluted by *Bi-Encoder CosSim* would be better than others with strict extraction constraints.

Empirically, while extraction-style prompts led to more deterministic behaviors, they substantially reduced to be more often to generate `CANNOTANSWER`. Consequently, despite better factual precision, these prompts exhibited lower recall and overall utility in open-domain retrieval-augmented generation (RAG) tasks. In contrast, Prompt1 preserved contextual reasoning ability, achieving the highest overall recall and better alignment with the reranker output.

Given `prompt1`'s robustness, we refined it by enhancing contextual guidance for reranking and adding structured reasoning cues to help the model prioritize relevance and factual consistency. The exact configuration is as Table 11This adjustment aimed to improve alignment between the retriever and reranker stages without increasing computational overhead.

Besides, the demonstrations are added to provides clear demonstrations of correct answers with complete sentence, part of one sentence, and `CANNOTANSWER`

outputs. (Taken from `train_val.txt`). Output length constraint also added to encourage concise responses.

The final configuration is shown in Table 11:

System Prompt	`You are a precise and context-grounded QA model.\n Your task is to answer questions **strictly based on the provided context passage\n\nFollow these rules carefully:\n1. Use information only from the context passages.\n2. If multiple passages are relevant, synthesize them **concisely** (25 words).\n3. When quoting or extracting, prefer the **exact phrasing** from the passage.\n4. If the answer cannot be found or inferred with HIGH confidence, output exactly\n5. Avoid adding explanations, paraphrasing, or external knowledge.\n6. Respond in one clear sentence only.\n\nExample 1:\nContext: "The album gained critical acclaim and achieved gold status."\nQuestion: "Was the album successful?"\nAnswer: "The album gained critical acclaim and achieved gold status."\n\nExample 2:\nContext: "Hackett starred as the title character on NBC-TV's Stanley, a 1956-57 situation comedy which ran for 19 weeks on Monday evenings at 8:30 pm EST."\nQuestion: "Was Stanley a character Buddy Hackett played?"\nAnswer: "Hackett starred as the title character on NBC-TV's Stanley, a 1956-57"\nExample 3:\nContext: "The game's art is made up of stick figure drawings."\nQuestion: "Did he write it in Vegas?"\nAnswer: CANNOTANSWER`
User Prompt	`Context Information:\n{context_text}\n Question:` {query}`\n\nAnswer:`

Table 11: Design of the (`Prompt 7`), which provides more examples, encourages concise extraction, and avoids from external knowledge or speculative inference.

The comparative performance is listed in Table 12 , where the `prompt7` and `prompt8` clearly outperforms the previous best configuration (`prompt1`) in cosine similarity, while maintaining strong MRR and retrieval recall.

Config.	Recall@10	MRR@10	CosSim
`prompt1`	0.8223	0.7088	0.3681
`prompt7`	0.8223	0.7088	**0.3853**

Table 12: Comparative performance of RAG prompt refinements. The latest `prompt7` achieves higher Cosine similarity while maintaining Recall@10 and MRR.

Report - Q3: Additional Analysis (2%)

Interesting Analysis Using Plots and Experimental Data from Inference (2%)

- Scores will be assigned based on the richness and depth of your analysis.

How Prompts Affect the Results

In Table 5, I observed that the abnormal proportion of generated "CANNOTANSWER" responses correlates with declining Bi-Encoder cosine similarity. To verify this observation, I tested `prompt7` (a hybrid approach), which generated **1051** CANNOTANSWER responses out of 3342 queries. This proportion is significantly closer to the actual distribution in the dataset.

A key question emerged: why do `prompt2` through `prompt6` increasingly favor CANNOTANSWER responses as we add more evaluation criteria?

To investigate further, I analyzed the reranker scores from `result.json`. The Top-1 reranker scores shown in Table 13 reveal a critical pattern: extraction-based prompts like `prompt6` establish a higher confidence threshold, while generative prompts like `prompt1` maintain a much lower acceptance threshold. This suggests that the model interprets low reranker scores as insufficient evidence for text extraction, leading to more CANNOTANSWER responses.

For `prompt7`, the hybrid approach appears promising and could potentially be improved by explicitly demonstrating when to output "CANNOTANSWER".

Table 13: Reranker Top-1 Scores vs. Generation Behavior. `Prompt1` employs a generative approach allowing the model to synthesize answers; `Prompt2` through `Prompt6` use extraction-based approaches requiring exact text copying with increasing strictness; `Prompt7` combines both strategies. "Avg Score (Cannot)" represents the average reranker score of the top-1 retrieved document when the model generates CANNOTANSWER, while "Avg Score (Valid)" shows the average for valid answers.

Prompt	**Cannot Count**	**Avg Score (Cannot)**	**Avg Score (Valid)**
1	0	-	1.745
2	2108	1.172	2.724
6	3116	1.612	3.575
7	1051	0.711	2.220

Do Scores Reflect True Confidence?

Although the model's performance is evaluated using reranker scores, a fundamental question remains: do these scores (computed from retrieved text) actually reflect the relevance of the gold answer? Specifically, is the gold answer always ranked highest?

To address this, I examined the rank positions of gold documents in the final reranked list. Table 14 reveals both the distribution of gold documents across ranks and their corresponding average scores.

Table 14: Gold Document Rank Distribution

Rank	**Count**	**Percentage**	**Avg Score**
Rank 1	2123	63.5%	2.256
Rank 2	552	16.5%	0.722
Rank 3	193	5.8%	0.090
Rank 4	117	3.5%	-0.208
Rank 5	46	1.4%	-0.625
Rank 6-10	108	3.2%	-
Not Retrieved	**594**	**17.8%**	**-**

Key Findings and Implications

These results raise significant concerns about the reliability of reranker scores as confidence measures. Only 63.5% of gold documents are ranked first, and 17.8% are not retrieved at all during the FAISS retrieval stage. Moreover, there is a striking score gap between Rank 1 (avg 2.26) and Rank 2 (avg 0.72), indicating substantial over-confidence in the top prediction.

While the trend suggests that reranker scores do correlate with relevance to some degree, they are not reliable confidence indicators. The sharp score drop between adjacent ranks implies that the score scale is poorly calibrated and does not reflect well-founded confidence levels. Although the reranker successfully distinguishes between relevant and irrelevant documents to some extent, its scores should not be directly interpreted as probabilities or confidence measures.

These findings indeed demonstrate that the reranker is currently insufficient for reliable answer retrieval. To improve performance, we would need either a better-calibrated scoring mechanism or a more sophisticated approach to document ranking that accounts for the complexity of relevance assessment.

References

[Esp22] Omar Espejel. *Train and Fine-Tune Sentence Transformers Models*. `https://huggingface.co/blog/how-to-train-sentence-transformers`. Hugging Face Blog. 2022.

[Kue24] Dragon Kue. *Mitigating False Negatives in Multiple Negatives Ranking Loss for Retriever Training*. `https://huggingface.co/blog/dragonkue/mitigating-false-negatives-in-retriever-training`. Hugging Face Blog. 2024.

Acknowledgement

I used ChatGPT, Claude, Qwen, and Gemini to assist with code development and report refinement. I also discussed with my classmates, especially regarding the strategy to train the reranker. Through this project, I realized that the assistance provided by AI tools is still limited, and I am grateful for the help from my classmate.